Encryption Framework

Practical work no. 1 - Free programming

SSAD course, USTHB, 2026-2027

Caesar, Hill and Playfair ciphers, password attacks and protection, message authentication

Group members	Task
Member 1	Caesar and Playfair ciphers
Member 2	Hill cipher and cipher attacks
Member 3	Passwords, attacks and protection
Member 4	Integrity (HMAC), interface, tests, report

1. What we built

The assignment asks for a small encryption framework with four parts: an interface, three classical ciphers, a study of password attacks with a way to defend against them, and one mechanism for authenticity or integrity. We wrote everything in Python 3 with the standard library only (Tkinter for the window, hashlib and hmac for hashing). The program runs with `python app.py` and the tests with `python tests.py`.

File	Content
ciphers.py	Caesar, Hill, Playfair, plus the attacks on them
passwords.py	The three password cases, dictionary and brute force attacks, PBKDF2 storage, lockout
integrity.py	SHA-256 digest and HMAC tag, seal and verify
app.py	The Tkinter interface, four tabs
tests.py	22 unit tests and a small benchmark (python tests.py bench)

The interface has four tabs. **Messaging** plays the story of the assignment: entity A types a message, picks an algorithm and a key, and sends it; the ciphertext and its authentication tag appear in a channel box that can be edited by hand to play the attacker; entity B verifies the tag and decrypts. **Cipher attacks** shows what an attacker can do against each cipher. **Password attacks** runs the dictionary and brute force attacks on the three password cases. **Protection** shows the defences and the time table.

2. Part 2 - Encryption and decryption

A sends a message to B. Both share the algorithm and the key. Everything is applied to the Latin alphabet, A to Z. For Hill and Playfair the text is first cleaned: upper case, letters only.

2.1 Caesar cipher

Each letter moves by n positions in the alphabet, to the right or to the left. With positions 0 to 25 the formula is `c = (p + n) mod 26` for a right shift and `c = (p - n) mod 26` for a left shift; decryption is the opposite shift. Case and punctuation are kept as they are. Because n is taken modulo 26, shifting by 29 is the same as shifting by 3.

Security flaws.

- Only 26 keys exist (25 useful), so trying every shift takes microseconds. The interface prints all 26 candidates and a human reads the right line immediately.
- Letter frequencies survive: the most common letter of the ciphertext is almost surely the image of E, which gives n directly. We implemented this guess and it finds the key on a sentence of a few dozen letters.
- Spaces and punctuation stay in place, so word lengths leak.

2.2 Hill cipher

The key is an n x n matrix K with integer entries mod 26. The text is cut into blocks of n letters, each block is a column vector p, and the ciphertext block is `c = K p mod 26`. Decryption uses the inverse matrix, `p = K^-1 c mod 26`. The inverse exists only when gcd(det K, 26) = 1, so the

program checks that and refuses bad keys with a clear message. The inverse is computed from the adjugate matrix and the modular inverse of the determinant (extended Euclid). If the text length is not a multiple of n it is padded with X. The key can be typed as numbers (3,3,2,5) or as a 4 or 9 letter word.

Worked example, key [[3,3],[2,5]], message HELP. Blocks HE = (7,4) and LP = (11,15). K(7,4) = (33,34) = (7,8) = HI and K(11,15) = (78,97) = (0,19) = AT, so the ciphertext is HIAT. This value is checked in the tests.

Security flaws.

- It is linear. With n known plaintext blocks and their ciphertext, K = C P^-1 mod 26 falls out of one matrix inversion. Our attack tab does this for a 2x2 key from only 4 known letters and recovers the exact key.
- A 2x2 key has only 157 248 invertible matrices, so brute force is easy for small n.
- Identical blocks give identical ciphertext blocks, which leaks repetition (it is a block cipher in ECB style).
- No diffusion between blocks: changing one block never affects the others.

2.3 Playfair cipher

A 5x5 square is built from the keyword (repeated letters removed) followed by the rest of the alphabet, with J merged into I. The message is split into pairs; a double letter inside a pair gets an X inserted, and an odd last letter is completed with X. For each pair: same row, take the letter to the right; same column, the letter below; otherwise swap the columns to form the rectangle. Decryption goes left and up. After decryption the program removes the X that were used as padding (an X between two identical letters and a trailing X), so B sees the original text; the removal is a heuristic, a real X in that position would be lost.

Check against the standard example: key 'playfair example', message 'hide the gold in the tree stump' gives BMODZBXDNABEKUDMUIXMMOUVIF, and the test suite asserts it.

Security flaws.

- It works on digrams, 600 possible pairs instead of 26 letters, which makes frequency analysis harder but not impossible. The digram count function shows the most repeated pairs; on a few hundred letters they match common English pairs (TH, HE, IN...).
- A pair and its reverse encrypt to reversed pairs (AB -> XY, BA -> YX). That regularity helps an attacker.
- No letter is ever paired with itself and letters are never encrypted to themselves; this narrows the guesses.
- J and I are merged and the padding X is ambiguous, so decryption can be slightly lossy.
- Hill climbing and simulated annealing break it on a few hundred letters.

3. Part 3 - Passwords

To make the experiments realistic the program does what a stolen database allows: it holds only a SHA-256 hash of the password, and the attacker tries candidates, hashes them and compares. Three cases were requested.

Case	Alphabet size	Length	Search space
Binary	2 (0,1)	3	2^3 = 8
Digits 1-4	4	4	4^4 = 256
Full set a-z A-Z 0-9 + * ...	74	3 (demo)	74^3 = 405224

The full set we use is 26 lowercase + 26 uppercase + 10 digits + 13 symbols (+ * - _ @ # $! ? % & =), which makes 74 characters. The assignment says 'etc.', so this choice is ours. The length for the third case is left to the user in the interface (1 to 4); we use 3 in the report because 4 characters already means about 29 million guesses.

3.1 The attacks

Dictionary attack. The attacker hashes a list of likely passwords (ours has about 50 entries: common passwords, short digit strings, a few local words) and compares. It costs at most as many hashes as the list is long, so it is the first thing to try. Measured: 'qwerty' is found after 4 guesses in 9 us. A random password such as 'Xk9+mQ' is not in the list and the attack ends after 47 guesses with nothing.

Brute force attack. Every combination of the alphabet is generated with `itertools.product` and tested. It always succeeds, the only question is the time.

Minimal time. For each use case the combined strategy is: dictionary first (cheap, covers weak human choices), then exhaustive search over the exact alphabet and exact length, which is the smallest possible space for the case. For the two tiny cases the whole space is smaller than the dictionary overhead, so the exhaustive search is simply instant. We also compare raw digest bytes instead of hex strings in the inner loop, which is faster.

Case	Space	Guesses (worst case)	Time
3 chars, 0/1	8	8	29 us
4 chars, 1-4	256	256	234 us
3 chars, full set	405224	405224	338.4 ms

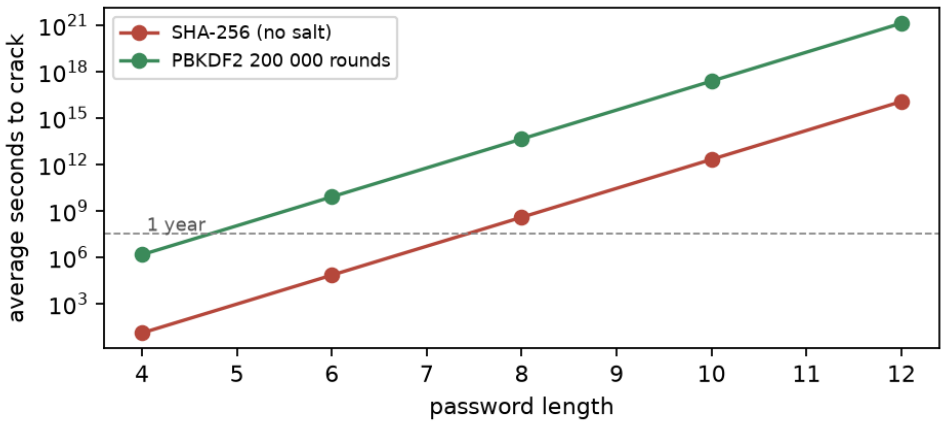
The machine used reached about 1167455 SHA-256 guesses per second in Python (a graphics card does billions per second, so real attackers are many thousands of times faster).

3.2 Protection

The attacks above succeed because the hash is fast and the alphabet or length is small. Our proposal has four layers; all of them are implemented in passwords.py.

- **Salted, slow hashing.** Passwords are stored as PBKDF2-HMAC-SHA256 with a random 16-byte salt per user and 200 000 iterations. The salt kills precomputed tables and makes two users with the same password look different. The iteration count makes each guess expensive: one check costs about 98.0 ms here, so the attacker drops from about 1167455 guesses per second to about 10, a slowdown of roughly 114402x.
- **Rate limiting and lockout.** The LoginGuard class locks the account for 30 seconds after 3 failures. An online attacker then gets at most 6 guesses per minute instead of thousands. The test suite checks that a correct password during the lock is still refused.

- **Password policy.** strength_problems rejects passwords shorter than 10 characters, those in the common list, and those missing a lowercase letter, uppercase letter, digit or symbol. This kills the dictionary attack and pushes the brute force space up exponentially.
- **Constant time comparison.** hmac.compare_digest is used so response time does not reveal how many bytes matched.



The graph shows the average time to crack a password made from the full 74 character set as the length grows. With no salt and a fast hash, 8 characters last about 12.2 years on our hardware; with the slow hash the same password lasts 1.4e+06 years. Each added character multiplies the work by 74.

Important remark: the two small cases (8 and 256 combinations) cannot be protected by hashing at all, since any attacker can try every possibility instantly even with a slow hash (256 x 0.14 s is about 36 seconds). The only defence there is the lockout and, of course, not using such passwords.

4. Part 4 - Authenticity and integrity

We chose one objective: **make B able to detect any modification of the message and know it comes from someone holding the shared secret.** The tool is an HMAC-SHA256 tag, computed over the ciphertext with a secret shared by A and B (encrypt-then-MAC).

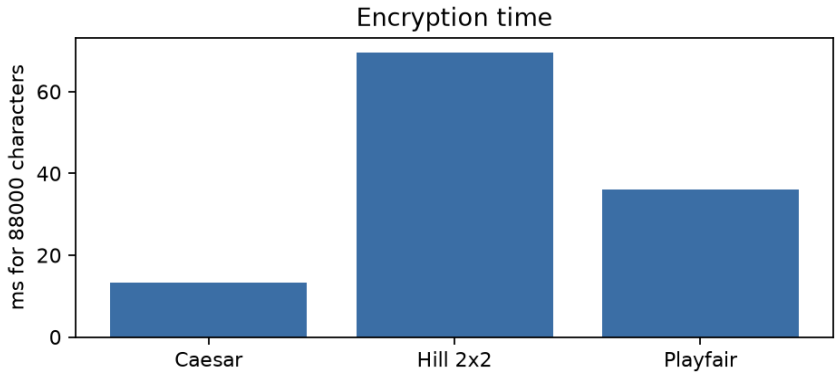
- A computes tag = HMAC(secret, ciphertext) and sends both.
- B recomputes the tag on what he received and compares in constant time. If it differs, the message is rejected before any decryption happens.
- In the Messaging tab, the 'Flip one letter' button simulates an attacker changing one letter on the channel; B then shows 'integrity check failed'. A wrong secret gives the same result, which shows authenticity.

Without this layer the classical ciphers have no integrity at all: in Caesar, flipping a ciphertext letter silently changes the matching plaintext letter. Note also that an HMAC proves the sender knows the secret but does not give non-repudiation; for that, digital signatures would be needed.

5. Tests and performance

The 22 unit tests in tests.py all pass. They cover: round trips in both Caesar directions for shifts 0 to 59, the known Caesar attacks, the Hill textbook example (key GYBNQKURP, ACT -> POH), 40 random Hill keys of size 2 and 3, rejection of a non invertible key, the known plaintext attack, the

Playfair reference vector, odd length messages, search space sizes, brute force on every one of the 8 binary passwords, the dictionary hit and miss, PBKDF2 verification and salt uniqueness, the lockout timing, and HMAC tamper detection.



Algorithm	Time for 88000 characters
Caesar	13.3 ms
Hill 2x2	69.6 ms
Playfair	36.2 ms

All three ciphers process about 90 kB of text in well under a tenth of a second, so speed is no concern. Hill is the slowest per character because of the matrix products, Caesar is the fastest. The real difference between them is not speed but resistance, covered in section 2.

6. Conclusion

The three ciphers work, round trip correctly and are checked against published examples, but each one falls to a known attack that our interface demonstrates: exhaustive key search or frequency for Caesar, known plaintext for Hill, digram statistics for Playfair. They are useful for learning and not for protecting real data. For passwords, small alphabets and short lengths are broken instantly whatever we do, and larger ones are only safe when combined with salting, a slow hash, a policy and lockout. Finally, the HMAC layer shows that confidentiality alone is not enough: integrity must be checked separately.